



AI ROUTE FINDER



AI-Powered Optimization



Python Implementation



A* Search Algorithm



Smart Navigation

Graph Search · Intelligent Systems

What is AI Route Finder?

Intelligent Route System

Finds an efficient route between a starting location and a destination using AI search.

Graph Representation

Locations are modeled as nodes; roads are weighted edges with distance costs.

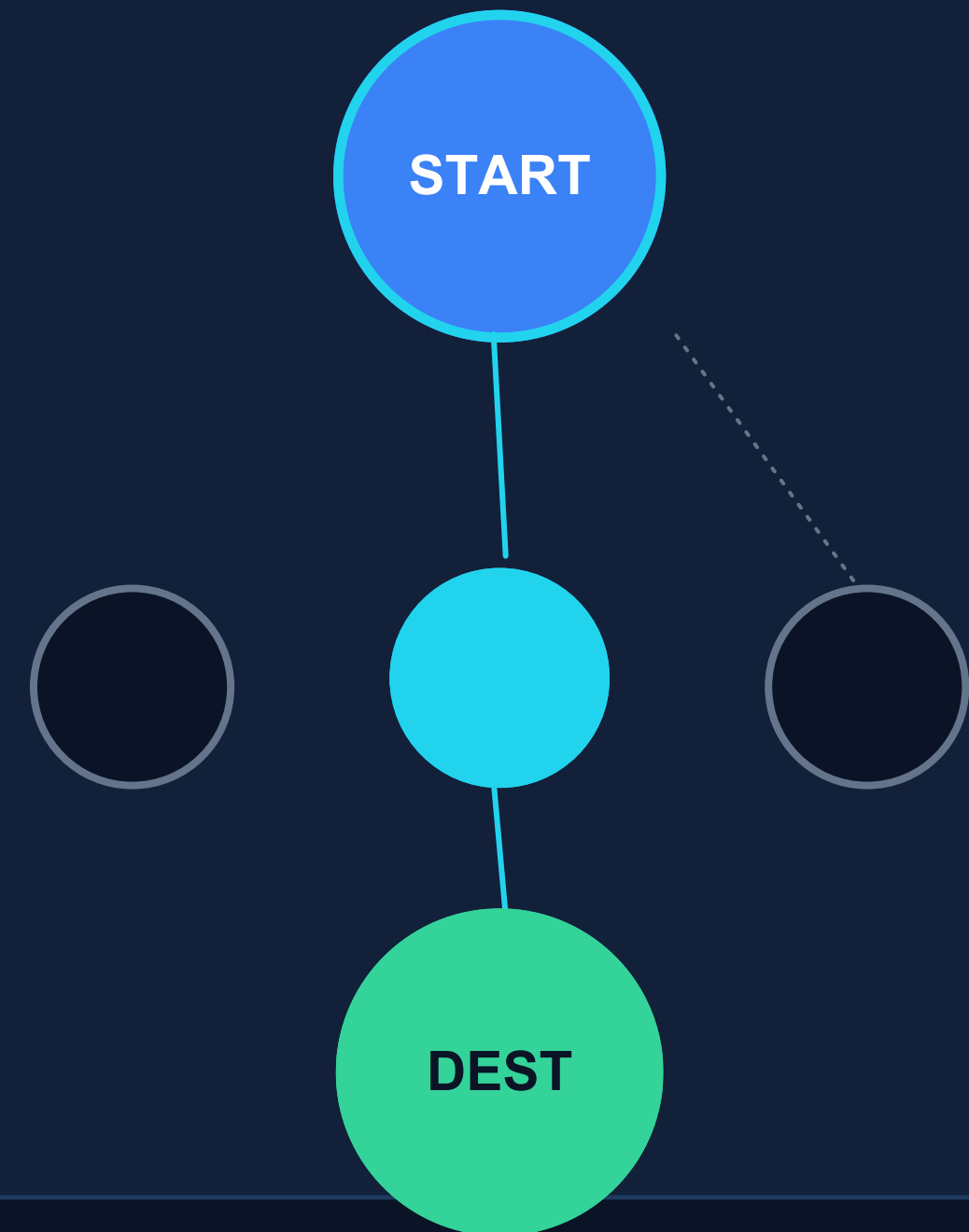
A* Search Algorithm

Uses the A* (A-star) algorithm to evaluate and select promising paths intelligently.

Extensible Design

Can be extended with traffic, GPS, weather, travel time, and road-condition data.

Route Selection Concept



Problem Statement & Objectives

PROBLEM

1. Multiple possible routes exist between any two locations.
2. Manually selecting a route is inefficient and error-prone.
3. Distance and cost are hard to compare without computation.
4. A systematic method is needed to calculate a suitable route automatically.

OBJECTIVES

- Automate route finding
- Reduce unnecessary distance
- Use graph-based search
- Implement A* algorithm in Python
- Display route & total distance
- Foundation for real-world nav systems

How the System Works



System Flow



Each stage feeds the next: raw locations become a weighted graph that A* explores until the optimal path is reconstructed.

Data Model: Locations & Roads

- locations dictionary stores latitude & longitude for each place
- roads dictionary stores connected locations with distance weights (km)
- The network behaves like a weighted undirected graph
- Bidirectional roads are stored in both directions

Python Data Structures

```
locations = {  
    "Chennai": (13.0827, 80.2707),  
    "Guindy": (13.0067, 80.2206),  
    "Tambaram": (12.9249, 80.1000)  
}  
  
roads = {  
    "Chennai": {"Guindy": 8},  
    "Guindy": {"Chennai": 8, "Tambaram": 12}  
}
```

Weighted Graph View



A* Search Algorithm

$$f(n) = g(n) + h(n)$$

Total estimated cost = Actual cost so far + Heuristic estimate to goal

$g(n)$

Actual cost from the starting node to the current node (sum of edge weights).

$h(n)$

Estimated cost from the current node to the destination (heuristic).

$f(n)$

Total estimated cost of the cheapest path through this node.

How A* chooses the next node

- ▶ A priority queue (min-heap) always expands the node with the lowest f-cost first.
- ▶ Lower f-cost nodes receive higher priority — the algorithm is optimistic and efficient.
- ▶ Search continues until the destination is dequeued; the path is then reconstructed via parent pointers.

Heuristic Function

Coordinate Storage

Latitude and longitude are stored for every location.

Difference Calculation

Δlat and Δlon are computed between two places.

Euclidean Approximation

Straight-line distance $\approx \sqrt{(\Delta\text{lat}^2 + \Delta\text{lon}^2)} \times 111$ km.

Role in A*

Guides the search toward the goal; admissible & consistent for this domain

Limitation

It is an approximation — not the true road distance.

heuristic() implementation

```
def heuristic(place1, place2):  
    lat1, lon1 = locations[place1]  
    lat2, lon2 = locations[place2]  
  
    dlat = lat1 - lat2  
    dlon = lon1 - lon2  
  
    return math.sqrt(  
        dlat**2 + dlon**2  
    ) * 111  
  
# ~111 km per degree  
# of latitude/longitude
```

Intermediate Python Code: Setup

```
import heapq
import math

locations = {
    "Chennai": (13.0827, 80.2707),
    "Guindy": (13.0067, 80.2206),
    "Tambaram": (12.9249, 80.1000),
    "Sholinganallur": (12.9010, 80.2279),
    "Kelambakkam": (12.8000, 80.2200),
    "Mahabalipuram": (12.6208, 80.1945),
    "Chengalpattu": (12.6810, 79.9888)
}

# roads dict defined next...
# (bidirectional weighted edges)
```

heapq

Priority queue for A* open set (min-heap on f-cost).

math

Used for sqrt in the Euclidean heuristic.

locations

7 places around Chennai with lat/lon pairs.

roads

Adjacency list with km distances; edges stored both ways.

Intermediate Python Code: A* Search

```
def a_star(start, goal):
    open_set = [] # priority queue
    heapq.heappush(open_set, (0, start))
    g_cost = {start: 0}
    parent = {start: None}

    while open_set:
        _, current = heapq.heappop(open_set)
        if current == goal:
            return reconstruct(parent, goal)

        for nbr, dist in roads[current].items():
            new_g = g_cost[current] + dist
            if nbr not in g_cost or new_g < g_cost[nbr]:
                g_cost[nbr] = new_g
                f = new_g + heuristic(nbr, goal)
                parent[nbr] = current
                heapq.heappush(open_set, (f, nbr))

    return None # no path
```

1 Open list
Min-heap ordered by f-cost

2 g_cost
Best known cost from start

3 parent
Enables path reconstruction

4 Expand
Pop lowest-f node

5 Relax
Update if cheaper path found

6 Push
Insert neighbor with new f

Intermediate Python Code: Input & Output

Program Flow

1. Display available locations
2. Get start location from user
3. Get destination from user
4. Validate both inputs exist
5. Call `a_star(start, goal)`
6. Print reconstructed route
7. Print total distance (km)
8. Print number of stops
9. Handle unreachable routes

Terminal Example

Available Locations:

Chennai Guindy Tambaram
Sholinganallur Kelambakkam
Mahabalipuram Chengalpattu

Enter Start: Chennai

Enter Destination: Mahabalipuram

Route:

Chennai → Guindy →
Sholinganallur → Kelambakkam →
Mahabalipuram

Total Distance: 73.00 km

Number of Stops: 4

Complete Program Structure

1 Graph Setup

locations dict + roads adjacency list with bidirectional km weights.

2 Heuristic

Euclidean distance $\times$ 111 km/degree using lat/lon pairs.

3 A* Search

Priority queue, g_cost, parent tracking, neighbour relaxation.

4 Reconstruction

Walk parent pointers from goal back to start, reverse list.

5 User Interaction

List places, read start/goal, validate membership.

6 Result Display

Print path arrows, total km, stop count; handle None path.

Benefits, Limitations & Real-World Example

BENEFITS

- Automatic route selection
- Efficient graph search
- Easy to understand & implement
- Python-based & extensible
- Great for learning AI search
- Integrable with real nav tech

LIMITATIONS

- No live traffic data
- No real-time closures
- Simplified road distances
- No actual GPS integration
- Heuristic is approximate
- Network is manually defined

EXAMPLE ROUTE

Chennai
↓
Guindy
↓
Sholinganallur
↓
Kelambakkam
↓
Mahabalipuram

Sample Path: Chennai → Guindy → Sholinganallur → Kelambakkam → Mahabalipuram · ~73 km

This intermediate project demonstrates the core ideas behind production navigation systems such as Google Maps and OpenStreetMap routers.

Conclusion & Future Enhancements

CONCLUSION

- AI Route Finder demonstrates intelligent route searching with Python and A*.
- Locations and roads are modeled as a weighted graph.
- A* combines actual cost $g(n)$ and estimated cost $h(n)$.
- The project is a solid foundation for smart navigation applications.

FUTURE ENHANCEMENTS

- Google Maps / OSM integration
- Live GPS & real-time traffic
- Travel-time & weather data
- Voice navigation & GUI / Web
- Mobile (Android) application
- ML-based route prediction



From Graph Search to Smart Navigation

A complete intermediate-level demonstration of A* pathfinding for real-world inspired routing.